

Design and Analysis of Algorithms (CS2009)

Date: Nov 6, 2025

Course Instructor(s)

Dr. Nasir Uddin, Dr. Kamran, Dr. Fahad Sherwani,
Mr. Sandesh, Mr. Kashif, Mr. Zulfiqar, Ms. Fizza
Mansoor

Mid-II Exam

Total Time (min): 75

Total Marks: 17.5

Total Questions: 4

Roll No

Section

Student Signature

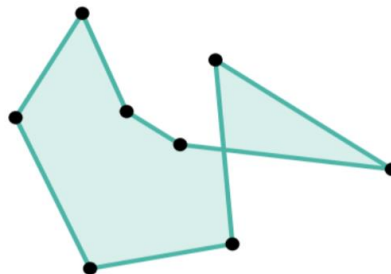
Do not write below this line

Attempt all the questions.

CLO #1: To apply acquired knowledge to solve computing problems complexities and proofs

Q1: A self-intersecting polygon (also called a complex polygon or crossed polygon) is a polygon where at least one of its edges crosses over another non-adjacent edge, as shown in the figure below.

[4 marks]



- a) [2.5 Point] Describe a complete, correct, brute-force algorithm to determine if the polygon P has any self-intersections. The polygon P is defined by a list of n vertices in order: $V_1, V_2, \dots, V_n$. The edges of the polygon are the segments connecting adjacent vertices (V_1, V_2) , (V_2, V_3) , ..., and (V_n, V_1) .
- b) [1.5 Points] What is the time complexity (in Big-O notation) of the brute-force algorithm you designed as a function of n (the number of vertices)?

Solution:

Algorithm:

```
function HasSelfIntersection(vertices):
```

```
  n = vertices.length
```

```
  // A polygon must have at least 4 vertices to self-intersect
```

```
  if n <= 3:
```

```
    return FALSE
```

```
  // Outer loop: Iterate through each edge (e1) from i=0 to n-1
```

```
  for i from 0 to n-1:
```

National University of Computer and Emerging Sciences

Karachi Campus

```
// Define the endpoints of the first edge (e1)
A = vertices[i]
B = vertices[(i + 1) % n] // (i+1)%n handles the wrap-around from Vn to V1
// Inner loop: Iterate through all following edges (e2)
// We start j from i+1 so we only check each pair once
for j from i + 1 to n-1:
    // Define the endpoints of the second edge (e2)
    C = vertices[j]
    D = vertices[(j + 1) % n]
    // --- Adjacency Check ---
    // We must skip edges that are adjacent to e1.
    // Case 1: Skip if e2 is the edge right after e1
    // (e.g., e1 is (V0,V1) and e2 is (V1,V2))
    // This happens when the start of e2 (j) is the end of e1 ((i+1)%n)
    if j == (i + 1) % n:
        continue
    // Case 2: Skip if e2 is the edge right before e1
    // (This handles the wrap-around case, e.g., e1 is (V0,V1) and e2 is (Vn-1,V0))
    // This happens when the end of e2 ((j+1)%n) is the start of e1 (i)
    if i == (j + 1) % n:
        continue
    // --- Intersection Test ---
    // If we are here, the edges are not adjacent.
    // We can now check if they properly intersect.
    if DoSegmentsIntersect(A, B, C, D) == TRUE:
        return TRUE // Found a self-intersection

// If the loops complete with no intersections found
return FALSE
```

The DoSegmentsIntersect Function's Logic

The DoSegmentsIntersect(A, B, C, D) function determines if segment AB properly crosses segment CD. This is done using the CCW (Counter-Clockwise) test.

A proper intersection occurs if and only if both of the following conditions are true:

1. Points C and D are on opposite sides of the line formed by AB.
 - We check this using: $CCW(A, B, C) \times CCW(A, B, D) < 0$.
 - (This ensures one CCW value is +1 and the other is -1).
2. Points A and B are on opposite sides of the line formed by CD.
 - We check this using: $CCW(C, D, A) \times CCW(C, D, B) < 0$.

If both of these products are negative, the segments "straddle" each other and must properly intersect. If either product is 0 or positive, they do not.

Complexity Analysis

Time Complexity: $O(n^2)$

Justification:

1. The algorithm is dominated by the nested loops.
2. The outer loop (for i) runs n times.
3. The inner loop (for j) runs, on average, $n/2$ times for each outer loop iteration.
4. This creates a total number of comparisons proportional to $n \times n$, which is $O(n^2)$.

National University of Computer and Emerging Sciences

Karachi Campus

5. All operations inside the inner loop (defining points, checking for adjacency, and calling DoSegmentsIntersect) are constant time, or $O(1)$. The CCW test is just a few arithmetic operations.

6. Therefore, the total time complexity is $O(n^2) \times O(1) = O(n^2)$.

CLO #3: To evaluate generic algorithmic solutions such as sorting, searching and graphs applied to real-world problems

Q2: Bob is implementing the “Find” functionality for a text editor. Users frequently search for repeated words or patterns in large documents, so the editor uses the KMP algorithm to make searching faster.

Given: **Text** = abababacaba ; **Pattern** = ababa

[4 marks]

- [1.5 Mark]** Compute the Longest Prefix Suffix (LPS) array for the given pattern.
- [2 Mark]** Show each step of the KMP matching process.
- [0.5 Mark]** Note the number of character comparisons made.

```

Algorithm KMPMatch(T, P)
    F ← failureFunction(P)
    i ← 0
    j ← 0
    while i < n
        if T[i] = P[j]
            if j = m - 1
                return i - j { match }
            else
                i ← i + 1
                j ← j + 1
        else
            if j > 0
                j ← F[j - 1]
            else
                i ← i + 1
    return -1 { no match }
    
```

Solution:

(a)

Index	0	1	2	3	4
P[i]	a	b	a	b	a

LPS = [0, 0, 1, 2, 3]

(b)

i=0, j=0, comparisons = 0

Step	i (Text)	j (Pattern)	T[i]	P[j]	Match?	Action	Comparisons
1	0	0	a	a	Yes	i=1, j=1	1
2	1	1	b	b	yes	i=2, j=2	2
3	2	2	a	a	yes	i=3, j=3	3
4	3	3	b	b	yes	i=4, j=4	4

National University of Computer and Emerging Sciences

Karachi Campus

5	4	4	a	a	yes	i=5, j=5	5
					j == M (pattern found)	Match at index 0; set j = LPS[4] = 3	—
6	5	3	b	b	yes	i=6, j=4	6
7	6	4	a	a	yes	i=7, j=5	7
					j == M (pattern found)	Match at index 2; set j = LPS[4] = 3	—
8	7	3	c	b	No	mismatch	j = LPS[2] = 1
9	7	1	c	b	No	mismatch	j = LPS[0] = 0
10	7	0	c	a	No	mismatch	i = 8
11	8	0	a	a	yes	i=9, j=1	11
12	9	1	c	b	No	mismatch	j=0
13	9	0	c	a	No	mismatch	i=10
14	10	0	a	a	yes	i=11, j=1	14
15	11	1	—	—	end of text	stop	—

Match found at indices: 0 and 2

(c)

Character Comparisons = 14

CLO #4: To construct and analyze real world problems solutions using different algorithms design techniques i.e. Brute Force, Divide and Conquer, Dynamic Programming, Greedy Algorithms.

Q3 :

[5.5 Mark]

- [1.25 Point]** While designing an algorithm to solve a computational problem when is Dynamic Programming used instead of Divide & Conquer?
- [1 Point]** Differentiate between Top-down and Bottom-up in dynamic programming (concise)
- [3.25 Point]** Your task is to create a single matrix by multiplying the following matrices, where you have to use each matrix. Find the minimum of multiplication operations to perform this task using an efficient algorithm. Also find the proper parenthesization.

Matrix A_1 with dimensions 20×30 ; Matrix A_2 with dimensions 10×20 ; Matrix A_3 with dimensions 40×50 ; Matrix A_4 with dimensions 30×40

Solution:

National University of Computer and Emerging Sciences

Karachi Campus

- If the problem can be solved by decomposing it into smaller subproblems and those subproblems are non-overlapping or independent then we use divide and conquer approach. If the subproblems are overlapping then we use dynamic programming.
- In Top-Down (Memoization), the problem is solved recursively, storing subproblem results as needed to avoid recomputation. In Bottom-Up (Tabulation), all subproblems are solved iteratively from the smallest up, building the final solution step by step. Top-down uses recursion and may need more memory, while bottom-up is iterative and generally more space-efficient

The matrixes can be multiplied in the following order:

$A_2 \times A_1 \times A_4 \times A_3$

	A_2	A_1	A_4	A_3
A_2	0	6000	18000 [1-4]	38000[4-3]
A_1		0	24000	90000 [1-4]
A_4			0	60000
A_3				0

Number of multiplication operation ($A_2 A_1$) = $10 \times 20 \times 30 = 6000$

Number of multiplication operation ($A_1 A_4$) = $20 \times 30 \times 40 = 24000$

Number of multiplication operation ($A_4 A_3$) = $30 \times 40 \times 50 = 60000$

Number of multiplication operation ($A_2 A_1 A_4$) = $\min((A_2)(A_1 A_4), (A_2 A_1)(A_4))$

Number of multiplication operation for (A_2)($A_1 A_4$) = $A_2 + A_1 A_4 + A_2 A_{14} = 0 + 24000 + 10 \times 20 \times 40 = 30000$

Number of multiplication operation for ($A_2 A_1$)(A_4) = $A_2 A_1 + A_4 + A_{21} A_4 = 6000 + 0 + 10 \times 30 \times 40 = 18000$

Number of multiplication operation ($A_1 A_4 A_3$) = $\min((A_1 A_4)(A_3), (A_1)(A_4 A_3))$

Number of multiplication operation for ($A_1 A_4$)(A_3) = $A_1 A_4 + A_3 + A_{14} A_3 = 24000 + 0 + 20 \times 40 \times 50 = 104000$

Number of multiplication operation for (A_1)($A_4 A_3$) = $A_1 + A_4 A_3 + A_1 A_{43} = 0 + 60000 + 20 \times 30 \times 50 = 90000$

Number of multiplication operation ($A_2 A_1 A_4 A_3$) = $\min((A_2 A_1 A_4)(A_3), (A_2 A_1)(A_4 A_3), (A_2)(A_1 A_4 A_3))$

National University of Computer and Emerging Sciences

Karachi Campus

Number of multiplication operation for $((A_2 A_1) A_4) (A_3) = A_{24} + A_3 + A_{24} A_3 = 18000 + 0 + 10 * 40 * 50 = 38000$

Number of multiplication operation for $(A_2 A_1)(A_4 A_3) = A_{21} + A_{43} + A_{21} A_{43} = 6000 + 60000 + 10 * 30 * 50 = 81000$

Number of multiplication operation for $(A_2)(A_1 A_4 A_3) = A_2 + A_{13} + A_2 A_{13} = 0 + 90000 + 10 * 20 * 50 = 100000$

Hence the optimal parathesization is $((A_2 A_1) A_4) (A_3)$

CLO #4: To construct and analyze real world problems solutions using different algorithms design techniques i.e. Brute Force, Divide and Conquer, Dynamic Programming, Greedy Algorithms.

Q4: An investor has a \$1,000,000 capital and is considering 4 potential projects. Each project requires a different initial investment and promises a different net profit. The investor can only invest in whole projects, not fractions. [4 marks]

Project	Investment Required (\$)	Net Profit (\$)
A	400,000	600,000
B	300,000	400,000
C	500,000	800,000
D	200,000	250,000

You are required to choose the projects, to get maximum profit, by applying the relatable Dynamic algorithm. Also find the projects that the investor should select. (Note: Do all steps)

Solution:

Capacity (w) -> 0 1 2 3 4 5 6 7 8 9 10

Project 0 [0,0,0,0,0,0,0,0,0,0,0]

Project A (4,6)[0,0,0,0,6,6,6,6,6,6,6]

Project B (3,4)[0,0,0,4,6,6,6,10,10,10,10]

Project C (5,8)[0,0,0,4,6,8,8,10,12,14,14]

Project D (2,2.5)[0,0,2.5,4,6,6.5,8.5,10,12,12.5,14.5]

Answer:

Maximum Profit = \$1,450,000

Projects: B, C, and D. Total Investment = 300k + 500k + 200k = \$1,000,000.

National University of Computer and Emerging Sciences

Karachi Campus

1. An R&D department has a \$10M budget for 6 research projects. However, some projects have dependencies:
 - Project D can only be selected if Project A is also selected
 - Project E and Project F cannot both be selected (they are mutually exclusive)
 - Project C requires Project B to be undertaken

Project	Cost (\$M)	Expected Value (\$M)
A	3	7
B	2	5
C	4	9
D	1	3
E	5	11
F	3	8